

Reviewer Pack — Noncommutative Probability Invariant for BP Read-Twice Compl...

Entry 99f896385624 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 20:57:31 UTC

Noncommutative Probability Invariant for BP Read-Twice Complexity

Entry ID: 99f896385624 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 20:57:31 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

[“For a given read-twice branching program P , define its free entropy as $F(P) := \sup\{\int \rho \log \rho \, d\mu \mid \rho \text{ is a positive probability measure on } P\text{'s state space and } \mu \text{ is a faithful tracial state on the algebra of } P\}$. The conjecture states that for all read-twice BPs P with n inputs and t levels, if $F(P) = O(n^\alpha \log(t))$, then the size of P is $O(2^{n+at})$ for some constant a .”, “This implies that high free entropy in the algebraic structure of P is correlated with large read-twice complexity.”, “Furthermore, we conjecture that there exist BPs for which $F(P)$ is a lower bound on the logarithm of their size.”]

Rationale (proposer’s reasoning):

[“Free probability provides a framework to study noncommutative random variables and has been used in other areas of mathematics. By applying this theory to BPs, we may uncover new structural properties that are not accessible through traditional complexity measures.”, “If the conjecture holds, it would establish free entropy as a valuable tool for understanding the inherent complexity of read-twice BPs, potentially leading to novel algorithms and lower bounds.”]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 615533ce8328b60e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all read-twice BPs P with n inputs and t levels, the ratio of the BP size to 2^{n+at} is less than or equal to a predefined threshold (e.g., 1.5) when $F(P)$ falls within $O(n^\alpha \log(t))$ with high statistical significance ($p\text{-value} \leq 0.05$).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability theory" AND "branching program read-once vs read-twice complexity" - "Invariant for BP read-twice complexity" AND free entropy - "lower bound on logarithm of size" AND branching program free entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i + 1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def free_entropy(P, t):
        # Placeholder function to compute free entropy
        # This is a dummy implementation and should be replaced with actual computation
        return random.random() * t
```

```

def bp_size(n, t):
    # Placeholder function to compute BP size
    # This is a dummy implementation and should be replaced with actual computation
    return 2 ** (n + t)

n = random.randint(5, 40)
t = random.randint(1, 10)
P = [[random.random() for _ in range(n)] for _ in range(n)]
F_P = free_entropy(P, t)
size = bp_size(n, t)

a = random.uniform(0.1, 1.0)
threshold = 2 ** (n + a * t) / size

if F_P <= n**a * math.log(t):
    conjecture_holds = size <= 2**(n + a*t)
else:
    conjecture_holds = False
    counterexample = "F(P) exceeds  $O(n^{\alpha \log(t)})$ "

return {
    "metric_name": "size_to_free_entropy_ratio",
    "metric_value": size / (2 ** (n + a * t)),
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"F(P) exceeds  $O(n^{\alpha \log(t)})$ \" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4c2c.py", line 79, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1dce4c2c.py", line 71, in run_trial
    "counterexample": counterexample
                   ^^^^^^^^^^^^^
```

UnboundLocalError: cannot access local variable 'counterexample' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15759
2	propose	ollama_remote	glm4:latest	0	0	11581
3	preregistration	ollama_remote	glm4:latest	0	0	6134
4	novelty	ollama_remote	glm4:latest	0	0	4657
5	novelty	ollama_remote	glm4:latest	0	0	5372
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12993
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34091
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26292
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12837
10	judge	ollama_remote	glm4:latest	0	0	57475

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 187191 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/99f896385624.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/99f896385624.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/99f896385624.tar.gz` (if generated)